PROJECT 31A — API SURFACE & SPECIFICATION BIBLE (DOCUMENT 5)

Version 1.0.0

Classification: CONFIDENTIAL — FOR INTERNAL DEVELOPMENT AND DUE DILIGENCE ONLY

---

1. API DESIGN PRINCIPLES & CONVENTIONS

Project 31A utilizes a modern, async-first REST API built on **FastAPI 0.111+**. The interface is designed to support high-throughput, low-latency client routing, strict type safety, and standardized envelope responses.

1.1 Standardized API Response Envelope

All API handlers return responses wrapped in a uniform JSON envelope to simplify frontend state parsing and error management.

```
{
    "success": true,
    "data": { ... },
    "error": null,
    "meta": {
      "request_id": "req_8f1a23b9d0",
      "timestamp": "2026-06-22T09:37:00Z"
    }
  }
```

If an exception occurs (caught by the global exception handler in `backend/src/seo_platform/main.py:L268-L302`), the structure transitions:

```
{
    "success": false,
    "data": null,
    "error": {
      "error_code": "RESOURCE_LIMIT_EXCEEDED",
      "message": "Ahrefs API usage quota limit hit.",
      "details": {
        "limit": 100,
        "current": 100
      },
      "retryable": false
    },
    "meta": {
      "request_id": "req_8f1a23b9d0",
      "timestamp": "2026-06-22T09:37:00Z"
    }
  }
```

1.2 Common Error Code Registry

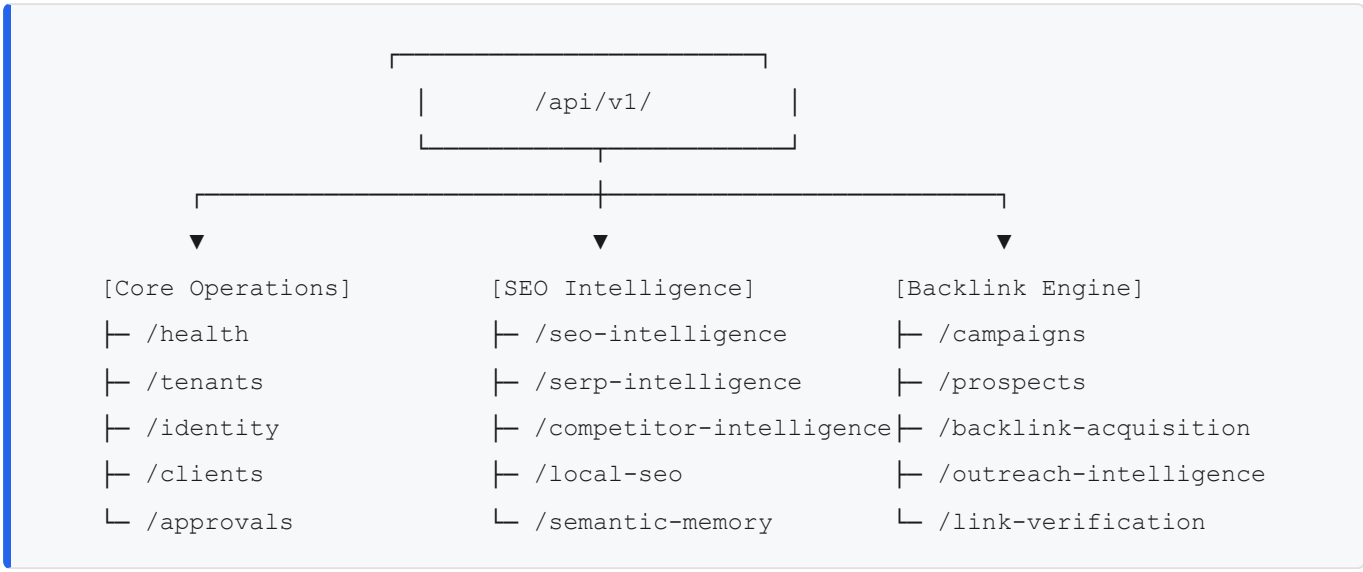
The platform defines semantic error codes to ensure uniform frontend error handling:

- `BAD_REQUEST` **(400):** Invalid body, formatting, or request variables.
- `UNAUTHORIZED` **(401):** Invalid or missing JWT bearer token.
- `FORBIDDEN` **(403):** Token valid but lacks permissions for client/tenant.
- `NOT_FOUND` **(404):** Target entity not found.
- `CONFLICT` **(409):** Unique constraints or status flow violation.
- `VALIDATION_ERROR` **(422):** Pydantic validation failures.
- `RATE_LIMITED` **(429):** API rate limits exceeded.
- `INTERNAL_ERROR` **(500):** Database errors, unhandled runtime exceptions.
- `TEMPORAL_UNAVAILABLE` **(503):** Temporal cluster timeout or worker pool crash.

---

2. API ROUTE INVENTORY (api/v1/)

All endpoints reside under the `/api/v1/` prefix. Routing imports and namespace mounting are centralized in `backend/src/seo_platform/api/router.py`.



2.1 Core Operational APIs

- `GET /health`: Direct endpoint at root. Returns health statuses of core infrastructure (PostgreSQL, Redis, Kafka, Temporal).
- `/api/v1/tenants`: Tenant management, invoicing settings, plan upgrades.
- `/api/v1/identity`: User profile resolution, role mappings, and JWT token checks.
- `/api/v1/clients`: Client CRUD operations. Triggering manual validation scans.
- `/api/v1/campaigns`: Campaign CRUD. Triggering campaign status changes.
- `/api/v1/approvals`: List approval queue, approve/reject prospects or templates.
- `/api/v1/plans`: Execution plans, showing step statuses and dependencies.
- `/api/v1/keywords`: Keyword database access.
- `/api/v1/kill-switches`: SRE tools to instantly pause operations.
- `/api/v1/recovery`: Failover and workflow recovery triggers.
- `/api/v1/audit`: Querying system and user activity logs.
- `/api/v1/alerts`: Manage notifications, critical events, and escalation rules.

2.2 SEO Intelligence APIs

- `/api/v1/seo-intelligence`: Main hub for keyword metrics, searches, and recommendations.
- `/api/v1/serp-intelligence`: Target SERP volatility metrics and domain overlap checks.
- `/api/v1/competitor-intelligence`: competitor domains overlap lists.
- `/api/v1/local-seo`: Local map pack positions and citation profiles.
- `/api/v1/semantic-memory`: Searching historical knowledge graphs and vectors.

2.3 Backlink Engine APIs

- `/api/v1/prospects`: List prospects, status overrides, email updates.
- `/api/v1/backlink-intelligence`: Link farm indicators, spam scoring, and DA checks.
- `/api/v1/backlink-acquisition`: List live links and monitoring metrics.

- `/api/v1/outreach-intelligence`: Inbox threads, replies, and templates.
- `/api/v1/link-verification`: Trigger link verification cron scan.

2.4 Citation APIs

- `/api/v1/citations`: Citation profiles and project status.
- `/api/v1/citations/automation`: Trigger browser-automation form-filling scripts.
- `/api/v1/citations/verification`: Email verification loop status for citation indexing.

2.5 Observability & SRE APIs

- `GET /metrics`: Prometheus metrics at root level.
- `/api/v1/observability`: SRE metric summaries.
- `/api/v1/queue-telemetry`: Realtime task queue processing depths.
- `/api/v1/realtime` / `/api/v1/realtime/sse`: Server-Sent Events stream for live client updates.

---

3. CORE ENDPOINT CONFIGURATIONS & SCHEMAS

3.1 POST `/api/v1/campaigns/` (Create Campaign)

- **Description:** Initializes a new backlink campaign.
- **Request Body (JSON):**

```
{
    "client_id": "d027f311-6b2c-473d-9ee1-45da9b8f2371",
    "name": "Acme Summer Skyscrapers",
    "campaign_type": "skyscraper",
    "target_link_count": 10,
    "config": {
      "min_da": 40.0,
      "max_spam": 0.25,
      "target_niche": "saas",
      "competitors": ["competitora.com", "competitorb.com"]
    }
  }
```

- **Response Shape:** Returns a `CampaignOut` object with status `draft`.

3.2 POST `/api/v1/campaigns/{id}/launch` (Start Campaign)

- **Description:** Triggers the campaign orchestration.

- **Process:**
  1. Validates status is `draft`.
  2. Submits workflow start request to Temporal client:
     - **Workflow:** `BacklinkCampaignWorkflow`
     - **ID:** `campaign-{campaign_id}`
     - **Task Queue:** `BACKLINK_ENGINE`
  3. Updates campaign database row status to `prospecting`.
  4. Returns `{ "success": true, "data": { "workflow_run_id": "run-8f12a" } }`.

3.3 GET `/api/v1/approvals` (List Approval Queue)

- **Description:** Fetches approval requests that are blocking workflow executions.
- **Response Shape:**

```
{
    "success": true,
    "data": [
      {
        "id": "app_23b9d027",
        "campaign_id": "c1a2f3b4-5512-421b-8ff1-1027f39b1a11",
        "approval_type": "prospect_approval",
        "status": "pending",
        "sla_deadline": "2026-06-25T12:00:00Z",
        "context": {
          "prospects_count": 42,
          "average_da": 52.4
        }
      }
    ]
  }
```

3.4 POST `/api/v1/approvals/{id}/decision` (Submit Decision)

- **Description:** Submits a decision to resume a blocked Temporal workflow.
- **Request Body (JSON):**

```
{
    "decision": "approved",
    "prospect_ids": ["uuid-1", "uuid-2"],
    "comments": "High quality list, proceed."
  }
```

- **Process:**
  1. Resolves `ApprovalRequest` record.
  2. Queries running campaign workflow details from DB (`workflow_run_id`).
  3. Sends Temporal Signal `approval_decision` (Gate 1) or `outreach_decision` (Gate 2) to workflow `campaign-{campaign_id}` containing the decision payload.
  4. Sets ApprovalRequest record status to `approved`.
  5. Returns `{ "success": true }`.

---

4. REALTIME SERVER-SENT EVENTS (SSE) STREAMING

To support reactive dashboard updates without heavy HTTP polling, the platform implements Server-Sent Events via FastAPI's `EventSourceResponse` in `backend/src/seo_platform/api/endpoints/realtime.py`.

- **Endpoint:** `/api/v1/realtime/sse`
- **Query Params:** `?tenant_id=uuid&client_id=uuid`
- **Flow:**
  1. Authentication header validated.
  2. Establishes SSE stream connection.
  3. Subscribes to Redis Pub/Sub channels matching `tenant:{tenant_id}` and `client:{client_id}`.
  4. When workers process activities or update workflow status, they publish events to Redis.
  5. The API handler reads events from Redis and flushes SSE payload:

```
event: campaign_progress
  data: {"campaign_id": "uuid", "status": "scoring", "completed_steps": 3, "total_step
  s": 11, "prospects_found": 42}
```

---

5. OPERATIONAL CONTROL: KILL SWITCHES & RATE LIMITING

5.1 Kill Switches API (`/api/v1/kill-switches`)

Used to instantly halt processes.

- **Available switches:** `email_sending`, `llm_inference`, `scraping_workers`.
- **Logic:**
  - `POST /api/v1/kill-switches/{name}/block` writes a key `killswitch:{name}:{tenant_id}` to Redis.

- `POST /api/v1/kill-switches/{name}/clear` removes the Redis key.
- Activities check Redis before executing operations. If blocked, they fail-fast or suspend execution.

5.2 Rate Limiting Architecture (`RateLimitMiddleware`)

Every API endpoint is protected by a Redis-backed token bucket rate limiter in `backend/src/seo_platform/api/middleware.py`.

- **Default limits:** `100` requests per minute per User.
- **Configuration:** Stored in `RateLimitConfig` model, letting admins increase limits for enterprise tenants.